

Frontend Engineering: Day 1

Architecture, HTTP, MVC, Jinja and Flask.

Sayan Chowdhury
Engineer- Data Analytics
Design Automation

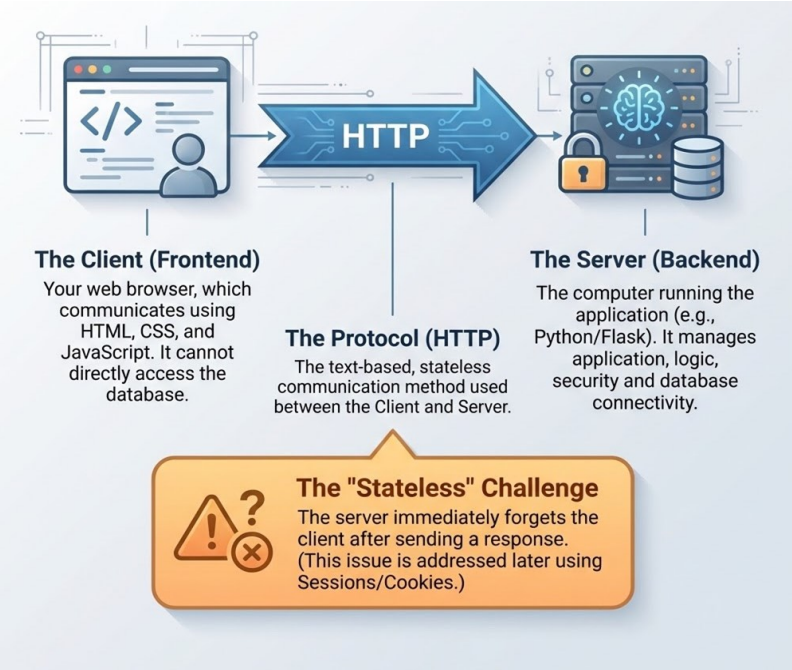
High-Level Architecture Overview

The Client (Frontend): Your web browser, which communicates using HTML, CSS, and JavaScript. It cannot directly access the database.

The Server (Backend): The computer running the application (Python/Flask). It manages application logic, security, and database connectivity.

The Protocol (HTTP): The text-based, stateless communication method used between the Client and Server.

The "Stateless" Challenge: The server immediately forgets the client after sending a response. (This issue is addressed later using Sessions/Cookies.)



The HTTP Request/Response Cycle

1. The Request (Client -> Server):

- Method: *GET* (Give me data), *POST* (Take my data), *PUT* (Update), *DELETE*.
- URL: The address (e.g., */dashboard/unit-1*).
- Headers: Metadata (*User-Agent*, *Content-Type*).
- Body: The actual payload (used mostly in *POST*).

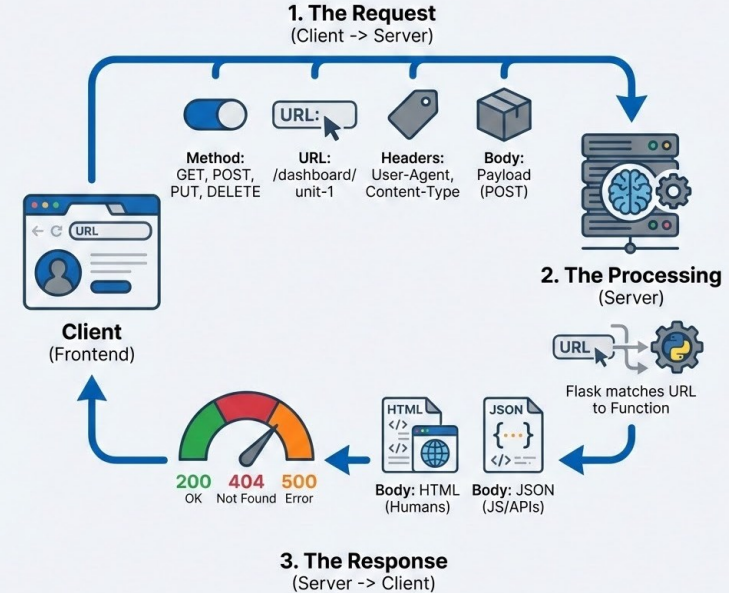
2. The Processing (Server):

Flask matches the URL to a Python function.

3. The Response (Server -> Client):

Status Code: 200 OK, 404 Not Found, 500 Server Error.

Body: Usually HTML (for humans) or JSON (for JS/APIs).



HTTP Headers – The Metadata of the Web

What are they?

Key-Value pairs sent before the body (payload).

Request Headers (Client -> Server):

Content-Type: "I am sending you JSON data" (*application/Json*).

Accept: "I want the response in HTML" (*text/html*).

Authorization: "Here is my ID badge" (*Bearer <token>*).

User-Agent: "I am using *Chrome/Python* Request Script."

Response Headers (Server -> Client):

Status: 200 (OK), 400 (Bad Request), 500 (Server Error).

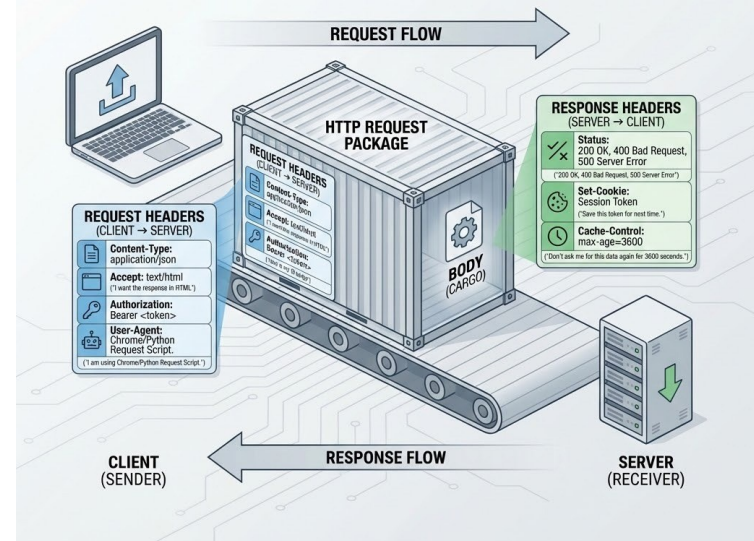
Set-Cookie: "Save this token for next time."

Cache-Control: "Don't ask me for this data again for 3600 seconds."

"If the Body is the cargo, Headers are the shipping label"

THE HTTP HEADERS: SHIPPING LABELS FOR DIGITAL CARGO

Body is the cargo, Headers are the shipping label.



What is REST? (Representational State Transfer)

REST (Representational State Transfer) is an architectural style for designing networked applications, primarily used to build web services (RESTful APIs) that allow systems to interact over HTTP

The 4 Constraints:

- **Client-Server Separation:** The UI (React/Jinja) and Data (Flask/SQL) can evolve independently.
- **Statelessness:** The server contains no client context between requests. Every request must contain *all* info (Auth tokens, params) needed to fulfil it.
- **Uniform Interface:** Using standard HTTP verbs (GET, POST, PUT, DELETE) predictably.
- **Cacheability:** Responses define if they can be stored locally to save bandwidth.

Treating the server as a collection of "Resources," not "Functions"

RESOURCE-BASED ARCHITECTURE & 4 CONSTRAINTS



CLIENT-SERVER SEPARATION



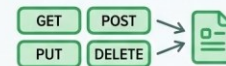
Evolve independently.

STATELESSNESS



No server context.
Every request has all info.

UNIFORM INTERFACE



Standard HTTP verbs predictably.

CACHEABILITY



Responses define if stored locally.

M-V-C Architecture (Model-View-Controller)

The Model (Data Layer):

Responsibility: Manages data, logic, and rules.

In our Project: The SQL Database and the Python Classes that represent it (e.g., class MaterialTest).

Rule: The Model never talks to the View directly.

The View (Presentation Layer):

Responsibility: Visualizing the data.

In our Project: The HTML files with Jinja2 variables (`{{ result }}`).

Rule: The View should be "dumb." It shouldn't calculate logic; it just displays what it's given.

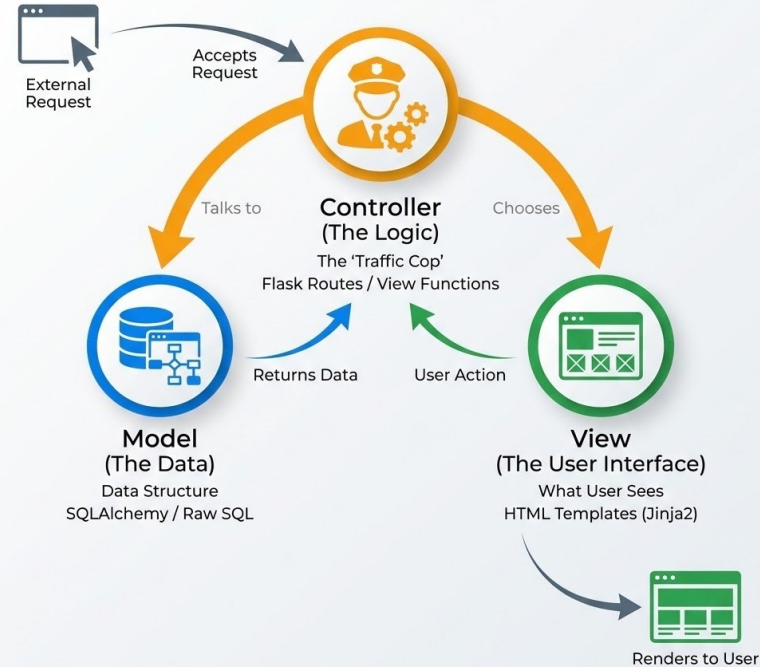
The Controller (Logic Layer):

Responsibility: Accepts input, converts to commands for Model or View.

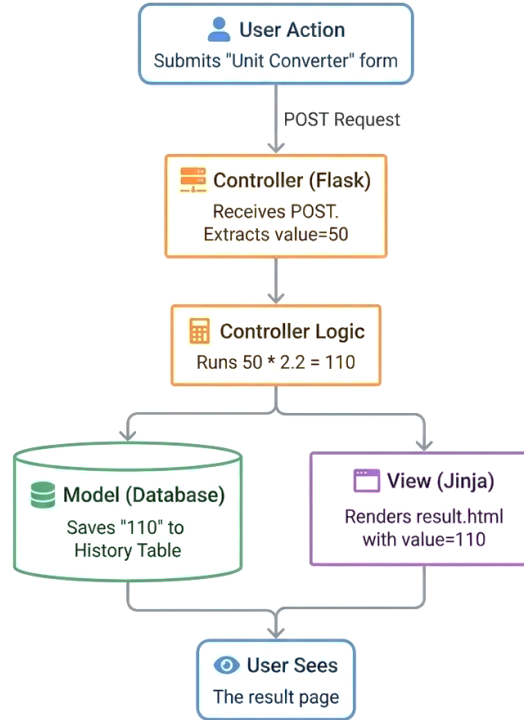
In our Project: Flask Routes (`@app.route`).

Action: User clicks "Calculate" -> Controller takes inputs -> Asks Model to save -> Asks View to show "Success".

The MVC Pattern in Flask



M-V-C Flow in Action (The "Round Trip")



What is Jinja2?

The Problem:

HTML files are hard-coded text. You cannot write `if user.is_logged_in` or `for item in database` inside a .html file.

The Solution:

Jinja2 is a Templating Engine. It lets you write Python-like logic inside your HTML

How it Works:

Template: HTML with special placeholders (e.g., `{{ user_name }}`).

Context: Python variables sent from Flask (e.g., `user_name="Engineer A"`).

Render: Jinja replaces the placeholders with real data -> Browser sees standard HTML.

"HTML is static. Python is dynamic. Jinja bridges the gap."

The Three Syntax Pillars

Syntax	Symbol	Usage	Example
Expressions	{{ ... }}	Print a value to the screen.	Welcome, {{ name }}
Statements	{% ... %}	Logic (Loops, If/Else). Doesn't print anything.	{% if user.is_admin %}
Comments	{# ... #}	Notes for developers. Not visible in browser source.	{# TODO: Fix mobile layout #}

Control Structures (Logic in HTML)

1. Conditionals (If/Else):

```
1 {% if battery_level < 20 %}↵
2 ....<div class="alert alert-danger">Warning: Low Battery!↵
   </div>↵
3 {% else %}↵
4 ....<div class="alert alert-success">System Nominal</div>↵
5 {% endif %}
```

2. Loop (for):

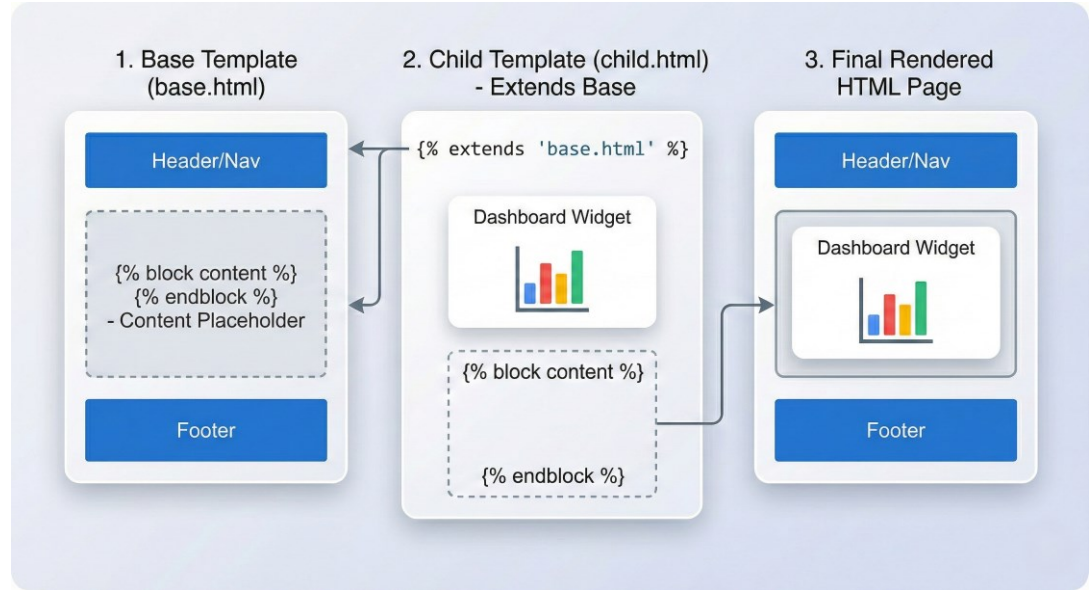
```
1 <ul>↵
2 ....{% for sensor in sensors %}↵
3 .....<li>{{ sensor.name }}: {{ sensor.value }}°C</li>↵
4 ....{% endfor %}↵
5 </ul>
```

Template Inheritance (The "Master Layout")

DRY Principle (Don't Repeat Yourself): "Define the Navbar once. Use it everywhere."

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Engineering App</title>
5 </head>
6 <body>
7   <nav>...</nav> <div class="container">
8     {% block content %}{% endblock %}
9   </div>
10  <footer>...</footer>
11 </body>
12 </html>
```

```
1 {% extends "base.html" %} {% block content %}
2   <h1>Dashboard</h1>
3   <p>System status is normal.</p>
4 {% endblock %}
```



Filters (Formatting Data)

Used to format data without changing the Python code.

Syntax: `{{ variable | filter_name }}`

Common Engineering Filters:

`{{ value | round(2) }}` -> Rounds 3.14159 to 3.14.

`{{ name | upper }}` -> Converts "pump_a" to PUMP_A.

`{{ list | length }}` -> Counts items (e.g., "5 active alarms").

`{{ default("N/A") }}` -> If variable is empty, print "N/A".

Why Flask?

“Full” vs. “Micro”:

Django (full): Comes with Auth, Admin, ORM pre-installed. Harder to customize for unique engineering needs.

Flask (Micro): Comes with nothing but routing and templating. You add exactly what you need.

Python Native:

Seamless integration with pandas, numpy, and scikit-learn (critical for engineering apps).

WSGI Compliance:

Built on Werkzeug (utility lib) and Jinja2 (templating).

Flask Implementation

```
1 from flask import Flask
2
3 # 1. Initialize the App
4 app = Flask(__name__)
5
6 # 2. Define a Route (The Controller)
7 @app.route('/')
8 def home():
9     # 3. Return a Response (The View -- temporary)
10    return "<h1>System Ready: Engineering Log v1.0</h1>"
11
12 @app.route('/status')
13 def status():
14    return "Operational"
15
16 # 4. Run the Server
17 if __name__ == '__main__':
18    app.run(debug=True)
```

Thank You